MEDIUM

ConflictingCandidateAndCertificate Misbehavior Stores No Cryptographic Evidence

Repo: ton-blockchain/ton **Branch:** testnet **Commit:** `3bb6abc` **Scope:** Primary — validator/consensus/simplex/ **Date:** 2026-03-14

Summary

Every call site that creates a `ConflictingCandidateAndCertificate` misbehavior report passes no arguments — the candidate and conflicting certificate are commented out in every caller. The misbehavior class stores nothing and produces an empty report. A Byzantine leader can propose candidates that conflict with existing certificates and face zero on-chain consequence, because the detecting node cannot prove the misbehavior to peers or to the elector contract.

The Factory and the Call Sites

The `ConflictingCandidateAndCertificate` class in `misbehavior.h` accepts no arguments:

```
// validator/consensus/simplex/misbehavior.h
class ConflictingCandidateAndCertificate : public Misbehavior {
 public:
  static MisbehaviorRef create() {      // no candidate or certificate parameter
    return td::make_ref<ConflictingCandidateAndCertificate>();
  }
  ConflictingCandidateAndCertificate() {}   // empty — nothing stored
};
```

There are four call sites in `pool.cpp`, all with the evidence arguments commented out:

```
// pool.cpp — line 646
return resolve_with(ConflictingCandidateAndCertificate::create(
    /* candidate, last_finalization_cert */));

// pool.cpp — line 656
return resolve_with(ConflictingCandidateAndCertificate::create(
    /* candidate, notarization_cert(slot) */));

// pool.cpp — line 666
return resolve_with(ConflictingCandidateAndCertificate::create(
    /* candidate, notarization_cert(first_nonfinalized_slot_ - 1) */));

// pool.cpp — line 676
return resolve_with(ConflictingCandidateAndCertificate::create(
    /* candidate, notarization_cert(slot) */));
```

The commented-out parameters show exactly what evidence should be captured. The detection logic is present and working; only the evidence collection is missing.

Conflict Scenarios Affected

Location	Detected Condition	Missing Evidence
pool.cpp:646	Candidate's parent slot already finalised (parent chain invalid)	Candidate + last finalization cert
pool.cpp:656	Candidate's slot already notarised with a different block	Candidate + notarization cert for slot
pool.cpp:666	Candidate's parent differs from the finalised parent at same depth	Candidate + notarization cert for last finalised slot
pool.cpp:676	Candidate's parent slot notarised with a different block	Candidate + notarization cert for parent slot

Impact

A misbehavior report without cryptographic evidence:

- Cannot be forwarded to other validators — they cannot verify the claim independently
- Cannot support an on-chain slashing transaction — the elector contract requires signed, verifiable proof
- Makes the detection purely local and silent — the wider network never learns of the misbehavior

A Byzantine leader can freely propose candidates that conflict with existing certificates. Each conflicting proposal is detected locally (resolver returns the misbehavior) but has no deterrent effect — the leader faces no penalty and the network gains no accountability signal.

Fix

Add candidate and certificate parameters to `ConflictingCandidateAndCertificate::create()` and store them. Implement serialisation so the report can be forwarded to other nodes and submitted on-chain:

```
// misbehavior.h — proposed fix
class ConflictingCandidateAndCertificate : public Misbehavior {
  CandidateId candidate;
  Certificate<NotarizeVote> cert;   // or FinalizeVote depending on conflict type
 public:
  static MisbehaviorRef create(CandidateId candidate, Certificate<NotarizeVote> cert) {
    return td::make_ref<ConflictingCandidateAndCertificate>(candidate, std::move(cert));
  }
  td::BufferSlice serialize() const override;   // implement for on-chain submission
};
```

Each call site should then pass the actual candidate and certificate rather than the commented-out placeholders.

TON Blockchain Contest Round 2 — Primary scope finding. Submitted by TesseraeVentures.